

Modelling Room Acoustics on a GPU

A new method that improves memory efficiency

Jacob Josiah Webber

Dissertation Presentation

Introduction

This talk aims to answer the following questions:

- ▶ Modelling acoustics: what and why?
- ▶ What is the current state-of-the-art?
- ▶ What is the new method proposed here?
- ▶ What were the results when this method was used?
- ▶ How could these be improved?

Sound

What is it?

From Wikipedia

“In physics, sound is a vibration that propagates as a typically audible mechanical wave of pressure and displacement, through a transmission medium such as air or water.”

From my undergraduate degree

$$\nabla^2 P - \frac{1}{c^2} \frac{\partial^2 P}{\partial t^2} = 0 \quad (1)$$

Room acoustics

- ▶ The physical properties of rooms affects how we hear sound.
- ▶ Sound reverberates off walls/objects.
- ▶ Waves diffract around corners.
- ▶ Some rooms echo, some sound dry.
- ▶ Backwards: If we know physical properties can we derive the sound?

The wave equation

$$\frac{\partial^2 P}{\partial t^2} = c^2 \left(\frac{\partial^2 P}{\partial x^2} + \frac{\partial^2 P}{\partial y^2} + \frac{\partial^2 P}{\partial z^2} \right)$$

where P is pressure deviation and c is speed of sound in medium.

- ▶ Very familiar to those who have studied physics
- ▶ It, and its application to acoustics, are derivable from first principles.
- ▶ Solving for P gives us acoustic properties of a system (boundary conditions are very important!).
- ▶ Solving this analytically for complex systems is not possible.

FTDT

Finite-difference time-domain method

Finite difference methods are a way of approximating and discretising differential equations so that they can be solved numerically.

Start off by replacing derivative with finite difference:

$$\frac{df(u)}{du} \approx \frac{f(u+h) - f(u-h)}{2h} \quad (2)$$

$$\frac{d^2f(u)}{du^2} \approx \frac{f(u+h) - 2f(u) + f(u-h)}{h^2} \quad (3)$$

Let h be the distance between samples (H) and P be a 3D grid

$$\delta_t^2 P_{l,m,p}^n \equiv \frac{1}{T^2} \left(P_{l,m,p}^{n+1} - 2P_{l,m,p}^n + P_{l,m,p}^{n-1} \right) \approx \frac{\partial^2 P}{\partial t^2} \quad (4)$$

$$\delta_x^2 P_{l,m,p}^n \equiv \frac{1}{H^2} \left(P_{l+1,m,p}^n - 2P_{l,m,p}^n + P_{l-1,m,p}^n \right) \approx \frac{\partial^2 P}{\partial x^2} \quad (5)$$

$$\delta_y^2 P_{l,m,p}^n \equiv \frac{1}{H^2} \left(P_{l,m+1,p}^n - 2P_{l,m,p}^n + P_{l,m-1,p}^n \right) \approx \frac{\partial^2 P}{\partial y^2} \quad (6)$$

$$\delta_z^2 P_{l,m,p}^n \equiv \frac{1}{H^2} \left(P_{l,m,p+1}^n - 2P_{l,m,p}^n + P_{l,m,p-1}^n \right) \approx \frac{\partial^2 P}{\partial z^2} \quad (7)$$

Taking the expanded 3D wave equation

$$\frac{\partial^2 P}{\partial t^2} = c^2 \left(\frac{\partial^2 P}{\partial x^2} + \frac{\partial^2 P}{\partial y^2} + \frac{\partial^2 P}{\partial z^2} \right) \quad (8)$$

This can be replaced with finite differences:

$$P_{l,m,p}^{n+1} - 2P_{l,m,p}^n + P_{l,m,p}^{n-1} = \frac{c^2 T^2}{H^2} (P_{l+1,m,p}^n + P_{l-1,m,p}^n + P_{l,m+1,p}^n + P_{l,m-1,p}^n + P_{l,m,p+1}^n + P_{l,m,p-1}^n - 6P_{l,m,p}^n) \quad (9)$$

Letting the Courant number be defined as

$$\lambda \equiv \frac{cT}{H} \quad (10)$$

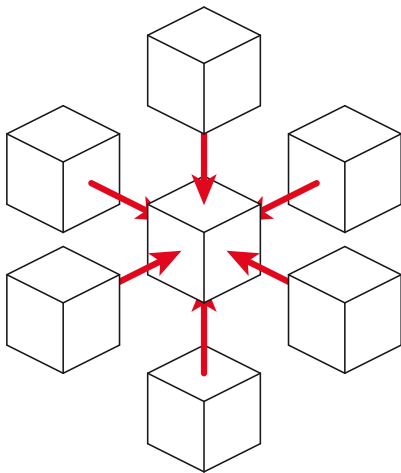
and our stencil

$$S \equiv P_{l+1,m,p}^n + P_{l-1,m,p}^n + P_{l,m+1,p}^n + P_{l,m-1,p}^n + P_{l,m,p+1}^n + P_{l,m,p-1}^n \quad (11)$$

gives a somewhat neater version of equation 9

$$P_{l,m,p}^{n+1} = (2 - 6\lambda^2)P_{l,m,p}^n + \lambda^2 S - P_{l,m,p}^{n-1} \quad (12)$$

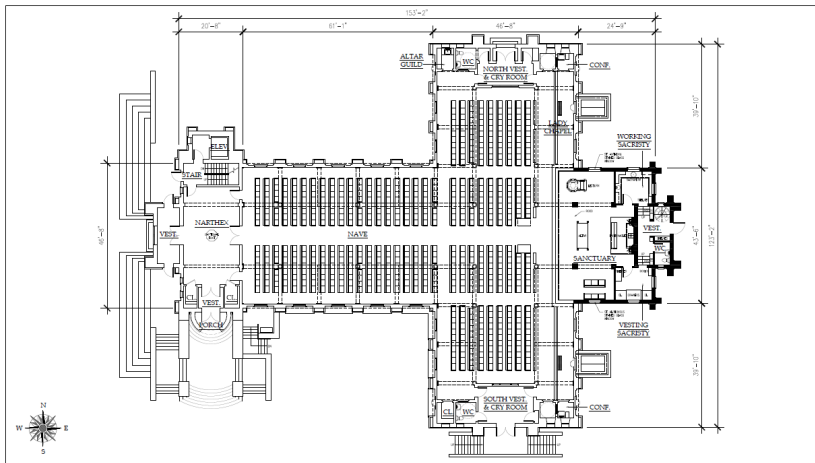
Stencil



Memory considerations

For stencilling operations, memory access is the bottleneck.

- ▶ Accurate models need a lot of points to model a room.
- ▶ Wavelength of sound at 20kHz is 17mm.
- ▶ High memory bandwidth means GPUs are used - limited memory.
- ▶ Standard 3d arrays are cuboid.
- ▶ Due to memory layout, neighbouring points in space will always be far apart in memory.



McCREERY Architects
100 Main Street, Suite 100
Rumson, NJ 07070
(201) 771-1100 www.mccreeryarchitects.com

HOLY CROSS CHURCH RUMSON, NEW JERSEY

Main Floor Plan
1/16" = 1'-0"
December 29, 2011
Sheet 4 of 21

The *Bounding Box*

The current method with irregularly shaped rooms

- ▶ An array of K values encodes the shape of the room.
- ▶ These are required to perform stencil with boundary conditions.
- ▶ Requires another array to be stored - can be chars.
- ▶ Store and operate on points that are outside the room - waste!

Stencil operation with these values becomes:

$$P_{l,m,p}^{n+1} = (2 - \lambda^2 K_{l,m,p}) P_{l,m,p}^n + \lambda^2 S - P_{l,m,p}^{n-1} \quad (13)$$

(Actual stencil is a bit more complicated and includes a wall permittivity term)

The *Bounding Box*

The current method with irregularly shaped rooms

Each CUDA kernel is allocated a block of the room the same size as its block of threads. Many of these blocks will be completely outside the room!

Data: Three dimensional arrays of $P_{l,m,p}^n$, $P_{l,m,p}^{n-1}$ and $K_{l,m,p}$

Result: Updated array of $P_{l,m,p}^{n+1}$

Get the 3D indices of the current thread from the block and thread IDs;

if $K_{l,m,p}$ *is not zero* **then**

 Compute the linear address from the 3D indices;

 Overwrite node $P_{l,m,p}^{n-1}$ with $P_{l,m,p}^{n+1}$;

end

Swap pointers.

Algorithm 1: CUDA kernel for update on an irregularly shaped room.

The Proposal

Use an array of structs instead!

- ▶ Store enough points for CUDA kernel to operate on.
- ▶ Only store structs with at least one point inside the room.
- ▶ Experiment with different sized arrays within the struct to see if caching benefits can be yielded.

The Struct

Listing 1: The block struct

```
typedef real bl_array[Bz][By][Bx];  
struct block {  
    int up;  
    int down;  
    int left;  
    int right;  
    int fore;  
    int aft;  
    bl_array u;  
    bl_array u1;  
    char k[Bz][By][Bx];  
};
```

Converting between representations

Going from bounding box to array of structs

Data: Three dimensional array of $K_{l,m,p}$ for a given room

Result: Array of blocks (AOB), block locations, number of internal blocks (T_i)

Divide $K_{l,m,p}$ into B_x by B_y by B_z sized blocks;

```
while not at end of blocks do
|   do nothing;
|   if block contains non-zero  $K_{l,m,p}$  value then
|   |   increment  $T_i$ ;
|   |   store block's location;
|   else
|   |   store zero as block location
|   end
|   go to next block;
end
allocate memory for  $T_i$  block structs;
while not at end of block locations do
|   if block location has non-zero value then
|   |   copy  $K_{l,m,p}$  values to correct block in AOB;
|   end
|   go to next block location;
end
```

Algorithm 2: Convert bounding box to array of blocks

Results

Two things are important:

- ▶ Memory capacity performance.
- ▶ Stencil performance.

Tests were run on three different shaped rooms.

But first: Correctness

For conversion algorithm: wrote a function that converts from bounding box coordinates to block index and array indices within block. Then loop through whole room to check K values. All good!

For stencil: Compare with Matlab script from AAG that implements bounding box method. All good!

To do: compare with real room or analytical solution to simple room.

Memory Capacity Results

Sphere

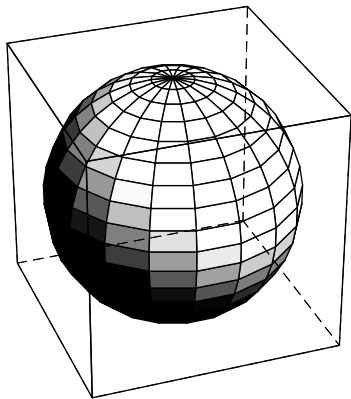


Figure: Computer graphic showing a sphere of unit diameter within a cube of unit side.

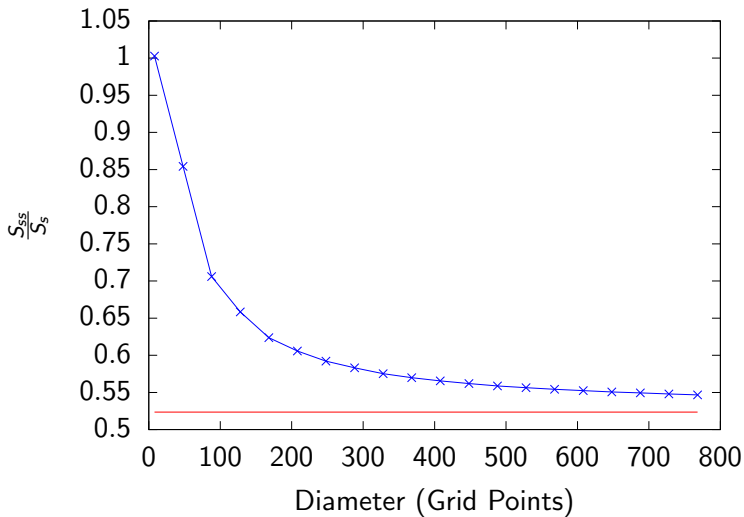


Figure: The fraction of memory used by the semi-structured approach with respect to the structured approach (double precision). The ideal fraction of $\frac{\pi}{6}$ is shown as a dashed line

Memory Capacity Results

Cube

- ▶ Forms the worst case scenario for AOS method.
- ▶ Integer indices of neighbouring blocks are extra overhead.
- ▶ This overhead is

Single Precision:

$$\frac{S_{ss}}{S_s} = 1 + \frac{24}{9 \cdot B_x \cdot B_y \cdot B_z} \quad (14)$$

Double Precision:

$$\frac{S_{ss}}{S_s} = 1 + \frac{24}{17 \cdot B_x \cdot B_y \cdot B_z} \quad (15)$$

Memory Capacity Results

Cube

- ▶ Sub-arrays of size $8 \times 8 \times 8$
- ▶ Yields $\frac{S_{ss}}{S_s} = 1.002757$
- ▶ This matches experimental results for cube for all sizes of room tested.

A worst case performance of an increase of approx 0.25% is not bad.

Memory Capacity Results

cross

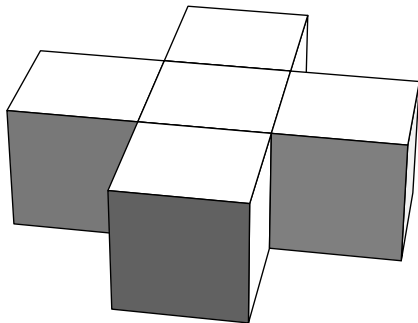


Figure: A cross made up of 5 equal cubes.

Memory Capacity Results

cross

- ▶ Bounding box would take 9 cubes.
- ▶ Cross takes 5 cubes.
- ▶ Ideal bounding box performance would take $\frac{5}{9} \cdot 1.002757$
- ▶ This was achieved performance in all cases tested.
- ▶ $\frac{S_{ss}}{S_s} = 0.557087$

Memory Capacity Results

Quantifying

Ideal is taken to be where two floating point numbers and one char is stored per grid point.

Room Shape	Diam	Ideal (MiB)	Achieved minus Ideal (MiB)		Improvement
			Structured	Semi-structured	
Cube	768	7344	0	20.25	N/A
Sphere	768	3845	3499	168.7	20.7X
Cross	488	9221	7736	226.4	34.2X

Table: Quantifying the memory capacity improvements by comparing unnecessary data stored between both methods.

Stencil Performance

No point in optimising for less memory if the result is slower than CPU code!

Room Shape	Diam	Memory Use (MiB)		Time (s)		Speed-up
		Structured	Semi-structured	Structured	Semi-structured	
Cube	608	3643.84	3653.90	79.49	81.25	0.98X
Sphere	608	3643.84	2012.92	79.50	44.78	1.78X
Cross	256	2448.00	1363.76	53.27	30.34	1.76X

Table: Performance results for the stencilling operation.

(It's not slower than CPU code)

Stencil Performance

It's a bit easier to see what's going on when you look at the data processing rates.

Room Shape	Data Rate (GiB s ⁻¹)	
	Structured	Semi-structured
Cube	44.77	43.92
Sphere	44.76	43.90
Cross	44.88	43.90
Mean	44.80	43.91

Table: Data processing rates.

To Do

3 months and 1500 lines of CUDA code later there is still plenty to do

- ▶ Check stencil with analytical solution with simple case
- ▶ Compare results with real room
- ▶ Optimize to take advantage of HBM2
- ▶ Port to OpenCL?

The End

This talk aimed to answer the following questions:

- ▶ Modelling acoustics: what and why?
- ▶ What is the current state-of-the-art?
- ▶ What is the new method proposed here?
- ▶ What were the results when this method was used?
- ▶ How could these be improved?